

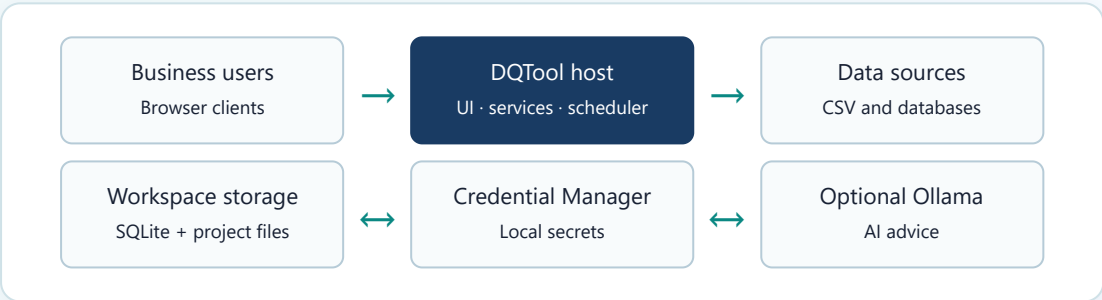
COMPLETE ARCHITECTURE HANDOVER

DQTool

Enterprise Architecture Brief

Visual executive overview plus the complete, unabridged technical architecture brief.

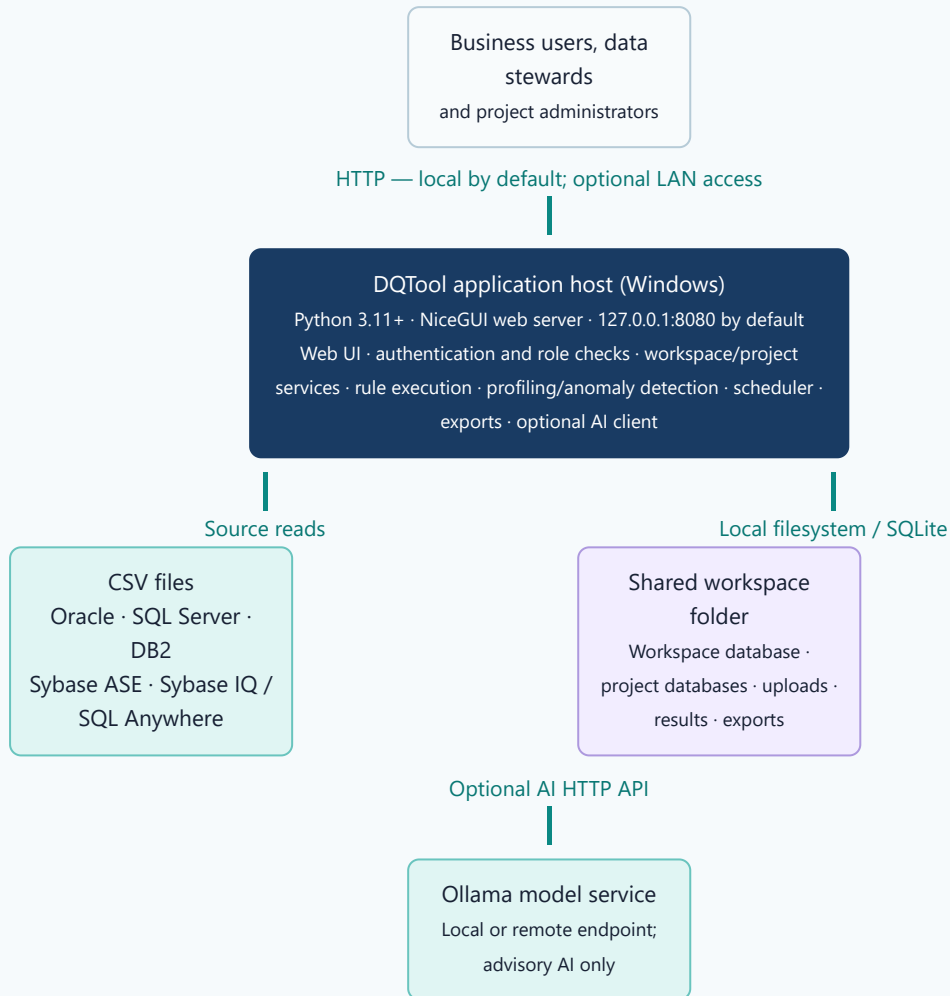
- Python 3.11+
- NiceGUI
- SQLite
- Windows
- Optional Ollama AI



Generated 13 August 2026 · The following appendix reproduces the complete technical content.

DQTool system and service flows

System context and container view



How data moves through DQTool

Interactive rule execution



The same execution service handles manual, batch, group and scheduled runs.

Profiling and anomaly detection



Profile suggestions remain advisory and require explicit user review before becoming rules.

Scheduled execution



Daily and weekly schedules use Europe/Brussels; timestamps are persisted in UTC.

Optional AI guidance



AI is optional and cannot create, run or schedule rules.

Complete architecture content

The sections below contain the full architecture brief in a clean, print-ready format. Text flow diagrams are represented visually in the preceding section.

++ DQTool enterprise architecture brief

This brief provides the technical information needed to produce DQTool context, container, deployment, data-flow, and security diagrams.

Application summary

DQTool is a Windows-first Python web application for defining and executing data-quality checks against CSV files and enterprise databases. It supports team workspaces and projects, role-based access, source profiling, anomaly and drift detection, rule scheduling, result evidence, Excel/CSV exports, and optional AI assistance.

It is currently a single deployable process: the NiceGUI web UI and background scheduler run in the same Python process.

Core container view

Visualized above. See the corresponding system or service-flow diagram in the visual architecture section.

Application components

Component	Responsibility	Implementation
NiceGUI web UI	Browser pages, user session, dashboards, preview, accounts, rules, results, schedules and profiling	web_app.py
Workspace service	Users, password verification, workspace/project registration, memberships and roles	workspace.py
Project/storage service	Project SQLite schema, migrations, and CRUD for rules, connections, groups, runs, schedules and profiles	project.py , storage.py
Connector service	Tests connections, discovers targets, previews data, executes source-specific queries	connectors.py
Execution service	Evaluates rules, persists metrics/status and writes bounded failed-row evidence	execution.py
Profiling service	Profiles columns, detects drift/content findings, identifies possible GDPR-sensitive data and derives editable rule ideas	profiling.py
Scheduling service	Calculates hourly/daily/weekly occurrences in Europe/Brussels time	scheduling.py
AI service	Calls Ollama for optional explanations and recommendations	ai.py
Export service	Produces failed-row CSV and anomaly/profile XLSX reports	anomaly_export.py

Integrations

Integration	Protocol / driver	Purpose	Direction
Browser clients	HTTP; NiceGUI session/cookies	Access the application	Inbound

Integration	Protocol / driver	Purpose	Direction
CSV files	Filesystem + DuckDB	Profiling, preview and rule evaluation	Read; browser uploads write files
Oracle	<code>oracledb</code>	Profiling, preview and rule execution	Read-only access recommended
SQL Server	ODBC via <code>pyodbc</code>	Profiling, preview and rule execution	Read-only access recommended
IBM DB2	ODBC via <code>pyodbc</code>	Profiling, preview and rule execution	Read-only access recommended
Sybase ASE	ODBC via <code>pyodbc</code>	Profiling, preview and rule execution	Read-only access recommended
Sybase IQ / SQL Anywhere	ODBC via <code>pyodbc</code>	Profiling, preview and rule execution	Read-only access recommended
Ollama (optional)	HTTP API	AI explanations and recommendations	Outbound
Cloudflare Access (optional)	HTTP request headers and service token	Protects a remote Ollama endpoint	Outbound

Native ODBC drivers must be installed on the DQTool host for SQL Server, DB2 and Sybase. Oracle uses the Python `oracledb` driver.

Storage and data locations

```
<workspace-root>/
dqtool_workspace.sqlite
<project-folder>/
  dqtool_project.sqlite
  uploads/
  results/
  exports/
```

Location	Contents	Considerations
<code>dqtool_workspace.sqlite</code>	Users, salted password hashes, projects, workspace/project memberships and roles	Back up with all project folders
<code><project>/dqtool_project.sqlite</code>	Connection definitions without passwords, rules, groups, schedules, run history and profiles	Back up with project files
<code><project>/uploads/</code>	CSV files uploaded through the browser; max 200 MB per file	May contain sensitive source data
<code><project>/results/</code>	Failed-row evidence from rule runs	May contain sensitive source data
<code><project>/exports/</code>	Generated CSV/XLSX reports	May contain sensitive source data
Windows Credential Manager	Per-user connection passwords and optional Cloudflare AI access tokens	Local to Windows host/user; not in SQLite

Location	Contents	Considerations
%LOCALAPPDATA%\DQTool\settings.json	Local application settings, including workspace selection, AI endpoint/model and session signing secret	Host-local configuration
NiceGUI session storage	Browser session state, including last-opened project	Not a system of record

An important design constraint is that database passwords are stored per Windows user in Credential Manager. In a central-server deployment, the server process identity controls which local credential store is used.

Data model

Workspace level

- User: username, salted PBKDF2-SHA256 password hash and workspace-admin status.
- Project: display name and project-folder reference.
- Membership: user-to-project assignment and project role.

Project level

- Connection: source type, host, port, service/database, driver, username, JSON configuration, owner and visibility.
- Rule: rule type, source reference, JSON configuration, owner and visibility.
- Rule group: hierarchical group with direct rule and child-group references.
- Schedule: target rule/group, cadence, enabled state, owner, last/next execution data.
- Rule run: status, timestamps, row counts, runtime, executor, optional schedule ID and failed-evidence path.
- Source profile: timestamped profile JSON keyed to source identity.

Rules and connection configuration are JSON. New keys must be optional when old projects are opened, and unknown keys should be preserved during edits.

Operational flows

Interactive rule execution

Visualized above. See the corresponding system or service-flow diagram in the visual architecture section.

Supported rules include not-null, uniqueness/duplicates, data volume, value range, regex, length, allowed values, date validity, freshness, custom SQL, referential integrity and keyed comparison. Custom SQL is authorised-user functionality and should be governed accordingly.

Profiling and anomaly detection

```
User selects a source
-> Profiling service reads source metadata/statistics
-> Profile snapshot persisted in source_profiles
-> Current snapshot compared with prior snapshot
-> Drift, content and GDPR review findings generated
-> User may export XLSX or review/edit suggested rules
```

Profiles include row count, data types, null/blank rates, distinct counts, min/max, averages, text lengths, frequencies, inferred meaning, and outlier/malformed-value signals. Rule ideas are advisory and never automatically saved.

Scheduled execution

Visualized above. See the corresponding system or service-flow diagram in the visual architecture section.

Schedules run hourly, daily or weekly. Daily/weekly timing uses `Europe/Brussels` ; timestamps are stored in UTC. Scheduling only occurs while the DQTool process is running.

Optional AI flow

Visualized above. See the corresponding system or service-flow diagram in the visual architecture section.

The default AI configuration is Ollama model `qwen3:8b` at `http://localhost:11434` . The endpoint and model are configurable. The normal AI flows send profile statistics, drift findings, GDPR categories and deterministic rule ideas, not source rows, sampled values or finding messages. AI is optional and does not run, save or schedule rules. A separate opt-in path can send sample values and requires explicit data-governance review.

Security architecture

- Local DQTool authentication; no built-in corporate SSO, LDAP, SAML or OIDC integration.
- Application passwords are salted PBKDF2-SHA256 hashes.
- Roles: Workspace Admin, Project Admin and Project User.
- Project objects support `private` , `shared` and specific-user sharing scopes.
- Database passwords and Cloudflare AI tokens are held in Windows Credential Manager, not SQLite or Git.
- Default bind address is `127.0.0.1` ; LAN hosting requires explicit host configuration and a Windows Firewall rule.
- The application should not be publicly exposed over HTTP.
- Use read-only source database accounts wherever possible.
- Uploaded CSVs, failed-row evidence and exports can hold sensitive data; apply filesystem ACLs, retention, backup and secure-disposal controls.

Deployment and configuration

Current deployment model

- Windows host.
- Python 3.11+ and the DQTool dependencies, or a packaged equivalent.
- One application process hosting UI and scheduler.
- A shared workspace folder available to the application process identity.
- Network reachability to browser clients, source databases and optionally Ollama.
- Native ODBC drivers installed for relevant database types.

Setting	Default / behaviour
<code>DQTOOL_HOST</code>	<code>127.0.0.1</code> ; set <code>0.0.0.0</code> for LAN access
<code>DQTOOL_PORT</code>	<code>8080</code> unless overridden
Workspace folder	Selected in application setup and stored locally
AI endpoint	<code>http://localhost:11434</code> by default
AI model	<code>qwen3:8b</code> by default

Setting	Default / behaviour
Schedule timezone	Europe/Brussels
Persisted timestamps	UTC

Constraints and enterprise considerations

- UI, scheduling, profiling and execution share one process.
- There is no message broker, queue, worker pool, API gateway, load balancer or independent scheduler component.
- SQLite is appropriate for a smaller shared/team deployment but should be assessed for concurrent writes, network-share locking, backup consistency and high availability.
- Horizontal scaling is not currently suitable: local credential storage, in-process scheduling and SQLite state create node affinity and duplicate-execution risks.
- For unattended scheduling, run the application as a managed Windows service or equivalent always-on process.
- The current design does not include built-in disaster recovery, central monitoring/alerting or SIEM/audit integration.

Diagrams to create

1. **System context diagram:** users, DQTool, source databases/files, Ollama, Windows Credential Manager and shared workspace storage.
2. **Container diagram:** single Python/NiceGUI process and its UI, workspace, connector, execution, profiling, scheduler, export and AI components.
3. **Deployment diagram:** Windows application host, workspace filesystem/SMB location, Credential Manager, ODBC drivers, local/remote Ollama, database network zones, browser clients and firewall boundaries.
4. **Data-flow diagram:** rule execution, profiling/anomaly processing, scheduled execution, CSV upload and AI metadata flow.
5. **Security/trust-boundary diagram:** browser/LAN, application host, workspace filesystem, credential store, source-data zone and optional AI endpoint; mark sensitive flows for credentials, CSV uploads, failed rows, exports and opt-in AI sample values.

Decisions for enterprise architecture

- Confirm whether DQTool is a local tool, departmental central server or enterprise platform.
- Decide whether to replace local authentication with OIDC/SAML/Active Directory.
- Decide whether secrets should move from Credential Manager to an enterprise vault.
- Assess replacing SQLite with a managed shared database for concurrency, audit, backup and high availability.
- Decide whether a shared filesystem is appropriate for uploads/results/exports, or whether object/document storage is required.
- Decide whether the in-process scheduler must be replaced with a durable enterprise job runner.
- Confirm whether AI metadata may leave the application trust boundary and define the approved model endpoint.
- Define retention, encryption, malware scanning, DLP and access-control controls for uploaded files, evidence and exports.
- Define authorisation, database identities, timeouts, audit controls and allowed-query policy for custom SQL.